Assignment 5: Perform encryption and decryption using columnar transposition and one-time padding cipher.

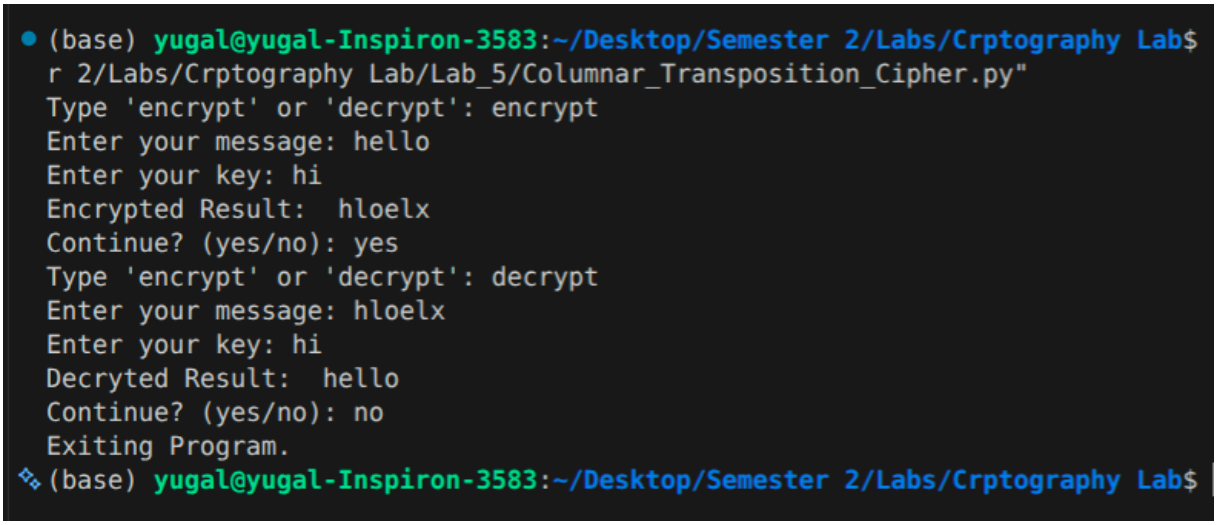
5.1 Implement Columnar Transposition Cipher in Python.

```python
import math

# Create Order
def get_key_order(key):
    key_list = list(key)
    sorted_key = sorted(key_list)
    order = []

    for ch in sorted_key:
        index = key_list.index(ch)
        order.append(index)
        key_list[index] = None # Avoid duplicate issues
    return order

# Encrypt
def encrypt(plain_text,key):
    plain_text = plain_text.replace(" ", "")
    cols = len(key)
    rows = math.ceil(len(plain_text)/cols)

    # Fill matrix row-wise
    matrix = []
    index = 0
    for r in range(rows):
        row = []
        for c in range(cols):
            if index < len(plain_text):
                row.append(plain_text[index])
                index += 1
            else:
                row.append('x') #padding
        matrix.append(row)
    order = get_key_order(key)

    cipher_text = ""
    for col_index in order:
        for r in range(rows):
            cipher_text += matrix[r][col_index]
    return cipher_text
```

```python

# Decrypt
def decrypt(cipher_text, key):
    cols = len(key)
    rows = math.ceil(len(cipher_text)/cols)

    order = get_key_order(key)

    # Create empty matrix
    matrix = [['' for _ in range(cols)] for _ in range(rows)] #list comprehension
    index = 0
    for col_index in order:
        for r in range(rows):
            matrix[r][col_index] = cipher_text[index]
            index += 1

    plain_text = ""
    for r in range(rows):
        for c in range(cols):
            plain_text += matrix[r][c]

    return plain_text.rstrip('x')

# Main Program
while True:
    choice = input("Type 'encrypt' or 'decrypt': ").lower()
    text = input("Enter your message: ")
    key = input("Enter your key: ")

    if choice == "encrypt":
        result = encrypt(text,key)
        print("Encrypted Result: ",result)

    elif choice == "decrypt":
        result = decrypt(text, key)
        print("Decryted Result: ",result)

    else:
        print("Invalid choice!")

    again = input("Continue? (yes/no): ").lower()
    if again == "no":
        print("Exiting Program.")
```

```
        break
```

Output



```
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$
r 2/Labs/Crptography Lab/Lab_5/Columnar_Transposition_Cipher.py"
Type 'encrypt' or 'decrypt': encrypt
Enter your message: hello
Enter your key: hi
Encrypted Result:  hloelx
Continue? (yes/no): yes
Type 'encrypt' or 'decrypt': decrypt
Enter your message: hloelx
Enter your key: hi
Decryted Result:  hello
Continue? (yes/no): no
Exiting Program.
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$
```

Figure 9: Program to implement Columnar Transposition Cipher

5.2 Implement One-Time Pad Cipher in Python.

```python
# Prepare Text
def prepare_text(text):
    clean_text = ""
    for c in text:
        if c.isalpha():
            clean_text += c.upper()
    return clean_text

# Encryption
def encrypt(plain_text, key):
    plain_text = prepare_text(plain_text)
    key = prepare_text(key)

    if(len(plain_text) != len(key)):
        return "Error: Key length must be equal to plaintext length."
    cipher_text = ""

    for i in range(len(plain_text)):
        p = ord(plain_text[i]) - ord('A')
        k = ord(key[i]) - ord('A')

        c = (p + k) % 26
```

```
        cipher_text += chr(c + ord('A'))
    return cipher_text

# Decryption
def decrypt(cipher_text, key):
    cipher_text = prepare_text(cipher_text)
    key = prepare_text(key)

    if(len(cipher_text) != len(key)):
        return"Error: Key length must be equal to ciphertext length."
    plain_text = ""
    for i in range(len(cipher_text)):
        c = ord(cipher_text[i]) - ord('A')
        k = ord(key[i]) - ord('A')

        p = (c - k) % 26
        plain_text += chr(p + ord('A'))
    return plain_text


# Main Program
while True:
    choice = input("Type 'encrypt' or 'decrypt': ").lower()
    text = input("Enter your message: ")
    key = input("Enter the key (same length as message): ")

    if choice == "encrypt":
        result = encrypt(text, key)
        print("Encrypted Text: ",result)
    elif choice == "decrypt":
        result = decrypt(text, key)
        print("Decrypted Text: ", result)
    else:
        print("Invalid Choice!")

    again = input("Continue? (yes/no): ")
    if again == "no":
        print("Exiting Program")
        break
```

Output

```
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$
r 2/Labs/Crptography Lab/Lab_5/One_Time_Pad_Cipher.py"
Type 'encrypt' or 'decrypt': encrypt
Enter your message: hello
Enter the key (same length as message): yugal
Encrypted Text:  FYRLZ
Continue? (yes/no): yes
Type 'encrypt' or 'decrypt': decrypt
Enter your message: FYRLZ
Enter the key (same length as message): yugal
Decrypted Text:  HELLO
Continue? (yes/no): no
Exiting Program
(base) yugal@yugal-Inspiron-3583:~/Desktop/Semester 2/Labs/Crptography Lab$
```

Figure 10: Program to implement One-Time Pad Cipher